

PORTFOLIO · AS-IS → TO-BE REDESIGN

AI 기반 데이터 표준 도메인 추천 시스템

레거시 데이터 거버넌스 솔루션의 'AI 도메인판별' 모듈을, 현재 기술스택으로 처음부터 재설계·재구현한 개인 포트폴리오 프로젝트

BACKEND Spring Boot 4 · JPA · JWT · Redis · OAuth2.0 · 외부 AI API

FRONTEND React · Redux(Reducer) · Redux-Saga · antd

Full-stack Junior Developer

개인 재구현 프로젝트 · 2026

회사 소스코드 미사용 · 아이디어와 아키텍처 패턴만 참고해 재구현 · 고객사 식별정보 미포함

프로젝트 개요

“

컬럼명을 분석해 AI가 표준 도메인
(코드·날짜·금액·연락처 등)을 추천하고,
담당자가 검토·승인하면 표준으로 확정되는 시스템

- ✓ 원본: 파견 근무 중 담당했던 데이터 거버넌스 솔루션의 일부 기능
- ✓ 회사 소스코드는 사용하지 않고 아이디어·아키텍처 패턴만 재구현
- ✓ 고객사명 등 식별정보는 포함하지 않음

왜 이 모듈인가

실무진이 봤을 때 'AI 활용 증명 + 레거시 이관 능력 + 이해하기 쉬운 스토리'가 동시에 드러나는 모듈로 선정

항목	내용
유형	개인 재구현 프로젝트 (원본: 파견 근무 중 담당했던 데이터 거버넌스 솔루션 일부)
Backend	Spring Boot 4 · Java 17 · JPA + MyBatis · Spring Security(JWT) OAuth2.0 Client · Redis · Oracle 18c XE
Frontend	React 17(Next.js 12) · Redux Toolkit(reducer) · redux-saga · antd
외부 연동	외부 AI API(RestClient) · CoolSMS · Email · PDFBox 리포트
산출물	리팩토링 설계서(.md) · ERD · DDL/DML/DCL(.sql, Oracle 18c XE)

왜 이 모듈을 포트폴리오로 선택했는가

담당 솔루션 전체(Java 약 19.8만 라인 · 5,200여 파일) 중 AI 도메인판별 모듈을 선정한 이유

1

'AI 활용' 역량이 코드로 증명됨

컬럼명을 형태소분석 · 토큰화해 피처를 만들고, 외부 추론 서비스에 확률 기반 후보를 요청해 저장 · 캐싱 · 승인까지 이어지는 엔드투엔드 파이프라인입니다.

2

레거시를 현재 스택으로 옮기는 능력

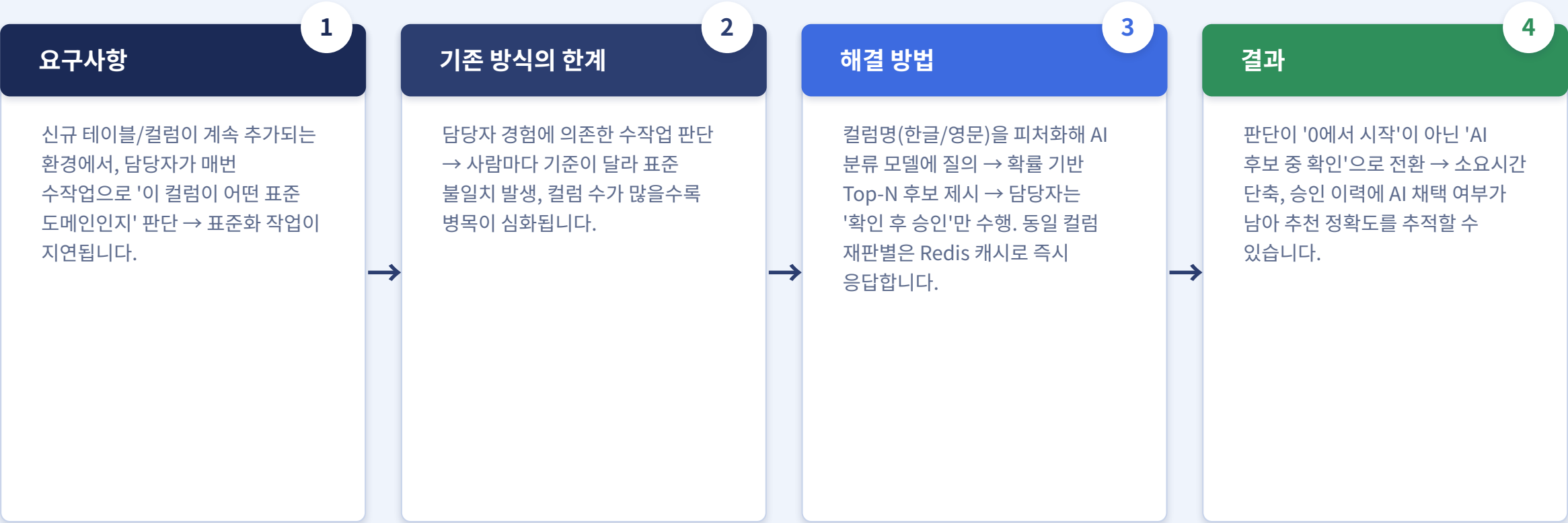
원본 Spring MVC 3.1 + MyBatis + JSP + gRPC + Quartz(셸 실행) 조합을 Spring Boot 4 + JPA/MyBatis + JWT/OAuth2/Redis + React로 옮기며 '왜 그렇게 바꿨는지'를 설계서에 남겼습니다.

3

'AI 추천 + 사람 승인', 이해하기 쉬운 스토리

'AI 추천 → 담당자 확인 → 확정' 흐름은 비개발자에게도 한 문장으로 설명되고, 내부적으로는 캐싱 · 인증 · 감사이력 같은 실무형 디테일이 채워져 있습니다.

요구사항 → 문제 → 해결 → 결과



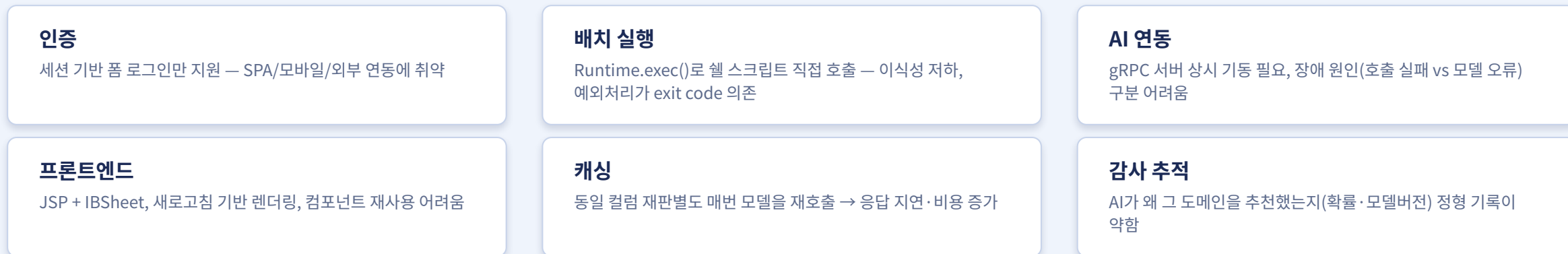
기술적으로 신경 쓴 점

배치 실행 상태를 INTERNAL_ERROR / CALL_ERROR로 분리해 장애 원인을 즉시 구분, JWT 로그아웃 시 Redis jti 블랙리스트로 즉시 무효화, 신청본/확정본 2단계 아이디어를 단일 신청 테이블 구조로 단순화

AS-IS — 원본 솔루션 아키텍처



AS-IS의 구조적 한계



TO-BE — 재구현 아키텍처

Frontend

React + Next.js

Page (antd) → Reducer → Saga

redux-saga로 예측 요청 폴링, 승인 API 호출 흐름을 제어

axios 인터셉터가 401 시 토큰 재발급을 자동 처리

→
Axios +
JWT

Backend

Spring Boot 4 (Java 17)

- Spring Security — JWT Filter + OAuth2 Client
- REST Controller → Domain Prediction Service
- AiDomainClassifier (RestClient → 외부 AI API)
- Spring Data JPA Repository
- MyBatis (대시보드 통계 · 동적 쿼리)

→

Redis

JWT 블랙리스트 · 판별 결과 캐시

Oracle 18c XE

표준 도메인 · 이력 데이터

외부 AI API

REST, .env로 API Key 관리

Email / SMS

승인 · 반려 결과 통보

Axios + JWT 인터셉터

401 응답 시 Refresh Token으로 Access Token 자동 재발급

RestClient → 외부 AI API

.env(spring-dotenv)로 API Key를 코드/리포지토리에 노출하지 않음

JPA + MyBatis 하이브리드

CRUD/연관관계는 JPA, 동적 조회 · 집계는 MyBatis로 역할 분리

성능/운영 개선 포인트 — 원본 대비

항목	AS-IS	TO-BE	기대 효과
재판별 응답속도	매 요청마다 gRPC 호출 (수백 ms 소요)	Redis 캐시 우선 조회	캐시 히트 시 응답시간 대폭 단축 (설계 목표 예시: 420ms → 12ms)
배치 기동 방식	셸 스크립트 Runtime.exec() (별도 프로세스)	Spring @Scheduled + DB 상태 관리	컨테이너 · 클라우드 이식성 확보
인증 방식	세션 기반 폼 로그인	JWT + OAuth2.0 + Redis 블랙리스트	Stateless API, 모바일/SPA 대응 로그아웃 즉시 무효화
오류 원인 구분	로그 텍스트로만 구분	INTERNAL_ERROR / CALL_ERROR 상태 분리	내부 로직 vs 외부 API 장애 즉시 파악
API 문서화	별도 문서 없음(엑셀 명세서)	springdoc-openapi 자동 생성	문서-코드 불일치 방지
AI 추천 신뢰도 측정	기록 없음	AI_SUGGESTED_YN 채택 여부 적재	추천 채택률을 정량 지표로 관리

핵심 기능

1

분석 대상 등록

테이블/컬럼 메타(물리명, 한글/영문 논리명, 데이터타입 등)를 등록하면 컬럼 단위 분석 대상으로 관리됩니다.

2

AI 도메인 추천

컬럼명을 피쳐화해 외부 AI API에 질의, ID/금액/날짜/명칭/번호/연락처/코드/플래그 등 후보를 확률과 함께 Top-N으로 제시합니다.

3

캐시 기반 응답 최적화

동일 컬럼 재판별은 Redis 캐시에서 즉시 응답, RESPONSE_MS·CACHE_HIT_YN을 기록해 개선 효과를 수치로 관리합니다.

4

승인 워크플로우

담당자가 AI 추천을 채택하거나 다른 후보를 신청하면 승인권자가 검토 후 승인/반려, 확정 표준에 반영하고 이메일/SMS로 통보합니다.

5

배치 재판별

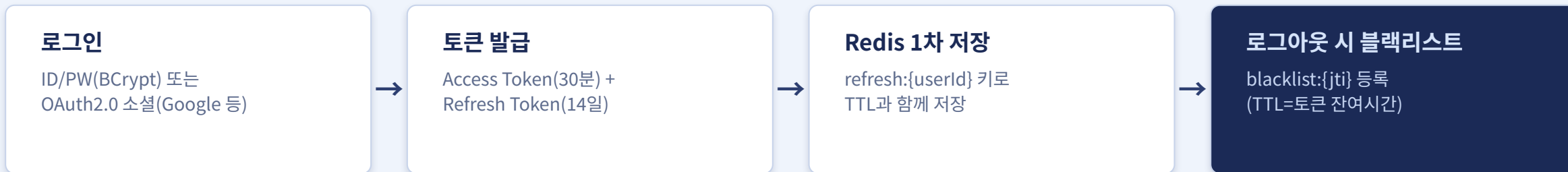
대량 컬럼을 배치로 등록해 비동기 재분석, 실행 이력을 5종 상태 (READY/RUNNING/SUCCESS/INTERNAL_ERROR /CALL_ERROR)로 관리합니다.

6

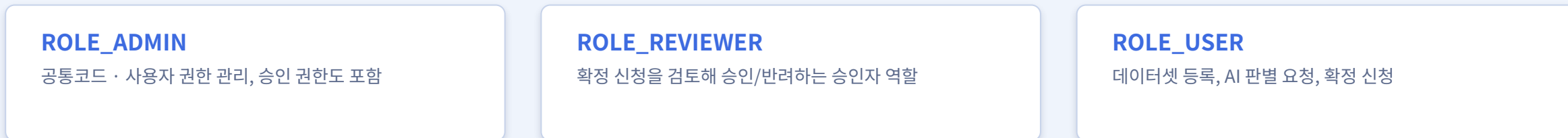
PDF 리포트

데이터셋 단위 판별 결과를 PDF로 내려받아 외부 보고용으로 활용합니다(Apache PDFBox).

인증/인가 — JWT + Redis + OAuth2.0



역할(Role) 기반 인가 — 3단계



승인(APPROVE) API는 @PreAuthorize로 ROLE_REVIEWER 이상만 호출 가능하도록 제한

JWT 한계 보완 — Redis 블랙리스트

JWT는 한 번 발급하면 회수가 어렵다는 한계가 있습니다. 로그아웃 시 Access Token의 jti를 Redis 블랙리스트(TTL=토큰 잔여시간)에 등록해, Stateless의 장점을 유지하면서도 즉시 무효화가 가능하도록 설계했습니다.

AI 도메인 판별 & 캐싱 플로우

POST /api/v1/columns/{columnId}/predict 요청 처리 시퀀스

1

React(Saga) → Controller

POST /predict 요청 (JWT 포함)

2

Service → Redis 캐시 조회

predict:{columnId} 키로 조회

3

[캐시 MISS] 피쳐 생성

DB 컬럼 메타 조회(JPA) → 형태소분석/토큰화로 피쳐 생성

4

[캐시 MISS] 외부 AI API 호출

RestClient로 POST /classify(API Key) → Top-N 확률 응답

5

[캐시 MISS] 결과 저장

DOMAIN_PREDICTION 저장(JPA) + Redis SETEX predict:{columnId} 3600

6

응답

예측 결과 DTO를 FE에 반환 (캐시 HIT 시 2번 이후 바로 이 단계로 단축)

캐시 HIT 시: 2번 조회 결과를 즉시 반환 → 3~5단계 생략, 평균 응답시간이 크게 단축됩니다.

승인 워크플로우 — 상태 다이어그램

원본의 '신청본/확정본 2단계' 아이디어를 상태값 기반 단일 신청 테이블 구조로 단순화



STANDARD_DOMAIN_REQUEST.AI_SUGGESTED_YN — AI 추천을 그대로 채택했는지 여부를 저장해, 이후 AI 추천 채택률(정확도)을 집계할 수 있는 설계

기술 스택 매핑표

package.json / build.gradle / .babelrc / .eslintrc 등록본 버전을 그대로 반영

Backend	
프레임워크	Spring Boot 4.0.7 (Java 17 toolchain)
ORM/매퍼	Spring Data JPA + MyBatis 4.0.1 (하이브리드)
보안/인증	Spring Security · JWT(jjwt 0.11.5) · OAuth2.0 Client
캐시/세션	Redis (Refresh Token · 판별 캐시 · 블랙리스트)
DB	Oracle 18c XE (ojdbc11)
외부 연동	RestClient(외부 AI API) · CoolSMS · Spring Mail · PDFBox
API 문서	springdoc-openapi-starter-webmvc-ui
환경설정	me.paulschwarz:spring-dotenv (.env)

Frontend	
프레임워크	Next.js 12 · React 17
상태관리	Redux Toolkit(reducer) + redux-saga
UI	antd 4.8 · @ant-design/icons
통신	axios(JWT 인터셉터) · jwt-decode
스타일	styled-components(ssr, displayName)
테스트	Jest · Testing Library · jest-environment-jsdom
린트	ESLint(airbnb, babel-eslint, ES2020)

버전은 프로젝트 등록본(package.json / build.gradle / .babelrc / .eslintrc)을 변경 없이 그대로 채택했습니다.

실제 구현 현황 — 설계로 그치지 않은 프로젝트

131개

백엔드 Java 파일

43개

프론트엔드 파일

v5.6

반복 개선 버전
(체인지로그 기준)

3단계

역할별 화면 분기
(관리자·승인자·일반사용자)

실전 트러블슈팅 사례 — 실행하고 고친 버그

jwt-decode v4부터 default export가 사라지며 decodeRoles()가 항상 빈 roles를 반환 → 관리자로 로그인해도 일반 사용자 화면만 보이던 버그. named import로 수정하고, 이름과 다르게 세로 사이드바로 구현돼 있던 MegaMenu를 가로형 메가메뉴로 재작성했습니다.

시연 시나리오 흐름

1

회원가입 · 로그인

2

데이터셋 · 컬럼 등록

3

AI 판별 실행

4

확정 신청 → 승인

5

관리자 권한 관리

트러블슈팅 & 기술적 의사결정

Q. 왜 JPA와 MyBatis를 같이 쓰나요?

A. 엔티티 CRUD/연관관계가 명확한 영역(사용자, 승인요청)은 JPA로, 조건이 자주 바뀌는 동적 조회·대시보드 집계는 MyBatis로 분리했습니다 — 쿼리 성격에 맞춰 도구를 선택했습니다.

Q. AI API가 느리거나 실패하면 어떻게 하나요?

A. 응답시간을 기록해 병목을 관찰하고, 실패는 CALL_ERROR로 분리 기록해 재시도(backoff) 정책을 적용할 수 있게 설계했습니다. 캐시가 있어 동일 요청이 몰려도 API 호출 자체를 줄입니다.

Q. JWT인데 로그아웃은 어떻게 즉시 반영하나요?

A. Redis 블랙리스트에 토큰의 jti를 잔여 만료시간만큼 저장해, Stateless의 장점을 유지하면서도 즉시 무효화가 가능하도록 했습니다.

CLOSING

이 프로젝트로 보여주고 싶은 것

- ✓ 대규모 레거시(약 20만 라인)를 스스로 읽고, 현재 스택으로 옮겼을 때 가장 설득력 있는 조각을 골라내는 능력
- ✓ CRUD를 넘어 외부 AI API 연동 · 캐싱 · 승인 워크플로우 · 배치 상태관리까지 하나의 완결된 흐름으로 설계하는 능력
- ✓ 기술 선택마다 '왜 그렇게 했는지'를 설계서로 남겨, 면접에서 근거 있는 답을 할 수 있는 근거

상세 설계서 · ERD · API 명세 · DDL/DML/DCL은 별첨 문서를 참고해 주세요

감사합니다